Université de Québec À Chicoutimi

6GEN723—Réseaux d'ordinateurs

Projet de conception : Guess Game

Objectifs :

- Amener les étudiants à appliquer les notions acquises en classe et en laboratoire pour concevoir et implémenter une application utilisant un réseau d'ordinateurs.
- Amener les étudiants à analyser et développer un protocole de communication.
- Familiariser les étudiants à la collaboration et au travail d'équipe dans le contexte du génie informatique.

Modalités :

1. Ce travail doit être réalisé en groupes de quatre personnes. Un membre de chaque groupe doit envoyer la liste des membres au professeur (**en mettant tous les membres en copie**) avant le **jeudi 19 mars 2026 à 23h59**. **Les groupes qui n'envoient pas leurs compositions avant cette date subiront une pénalité de -10%**.
2. L'évaluation de ce travail, au niveau du groupe, se fera à base du fonctionnement du code, la qualité de la démonstration. L'évaluation du travail de chaque membre du groupe se fera à base **de son rapport individuel** et de ses contributions dans la réalisation du travail ainsi que les échanges durant la démonstration.
3. La démonstration des projets se fera durant la semaine 16 comme mentionné dans le plan de cours. Des créneaux de démos vous seront communiqués pour organiser les réservations, **les démos se dérouleront lundi 20 avril 2026 de 10h30 à 12h00 et mardi 21 avril 2026 et jeudi 23 avril 2026 durant les heures de laboratoire.**

1 Introduction

Les jeux en ligne reposent sur des communications réseau permettant à plusieurs joueurs d'interagir à distance. La conception d'un protocole de la couche application pour ce type d'application doit tenir compte de plusieurs contraintes, notamment la gestion des connexions des joueurs, la découverte des parties disponibles, la synchronisation des actions entre participants et la fiabilité de l'échange des messages. Dans une architecture client-serveur classique, le serveur agit comme un point central de coordination : il permet aux joueurs de se découvrir, de créer des sessions de jeu et d'organiser les interactions initiales. Afin de réduire la charge du serveur et la latence pendant la partie, certaines architectures permettent ensuite aux clients de communiquer directement entre eux.

Le jeu proposé, Guess Game, est un jeu de devinette de combinaison de couleurs. Un joueur choisit une combinaison secrète composée de quatre couleurs. Les couleurs possibles sont : Rouge, Vert, Bleu, Jaune et Orange. Les autres joueurs tentent de deviner cette combinaison. Après chaque proposition, le joueur qui détient la combinaison secrète renvoie deux informations : le nombre de couleurs de la proposition présentes dans la combinaison secrète, et le nombre de couleurs correctement placées. Les joueurs disposent d'un nombre limité de tentatives, défini lors de la création de la partie. Un joueur qui devine exactement la combinaison secrète gagne la partie.

2 Comportement du serveur

Le serveur agit comme un service de coordination permettant aux joueurs de se connecter, de découvrir des salles de jeu et d'en créer de nouvelles. Le serveur utilise le protocole **TCP** au niveau de la couche transport et se comporte comme suite :

- Lorsqu'un client se connecte, il envoie le message : `GG|CONNECT|nom_joueur` .Le serveur enregistre le joueur et répond : `GG|CONNECTED|nom_joueur`
- Pour créer une salle de jeu, un client envoie : `GG|CREATE_ROOM|nom_salle|max_joueurs|max_tentatives` . Le serveur crée la salle correspondante et désigne le créateur comme administrateur de la salle. Il répond : `GG|ROOM_CREATED|nom_salle`
- Un client peut demander la liste des salles existantes avec le message : `GG|LIST_ROOMS` . Le serveur renvoie alors : `GG|ROOM_LIST|salle1,salle2,...`
- Pour rejoindre une salle, un client envoie : `GG|JOIN_ROOM|nom_salle` . Le serveur ajoute le joueur à la salle et répond : `GG|JOINED_ROOM|nom_salle|liste_joueurs`
- Un joueur peut quitter une salle avec : `GG|LEAVE_ROOM|nom_salle` . Le serveur confirme avec : `GG|LEFT_ROOM|nom_salle`
- L'administrateur d'une salle peut expulser un joueur avec : `GG|KICK_PLAYER|nom_salle|nom_joueur` .Le serveur supprime alors ce joueur de la salle et envoie : `GG|PLAYER_KICKED|nom_joueur`
- Lorsqu'un joueur souhaite démarrer une partie dans une salle, il envoie : `GG|START_GAME|nom_salle` . Le serveur informe tous les joueurs que la partie commence et leur transmet les informations nécessaires pour établir une communication directe entre eux : `GG|GAME_STARTED|nom_salle|liste_joueurs`
- Le protocole permet également de jouer contre le serveur. Dans ce cas, le client envoie : `GG|PLAY_SERVER|tentatives` .Le serveur choisit alors une combinaison secrète aléatoire et répond : `GG|SERVER_GAME_STARTED|tentatives`

3 Comportement du client

Le client est responsable de l'interface utilisateur, de l'envoi des commandes au serveur et de la communication directe avec les autres joueurs pendant la partie. Le comportement du client est comme suite :

- Lorsqu'un client démarre, il envoie le message :`GG|CONNECT|nom_joueur` .
- Après réception du message `GG|CONNECTED`, le joueur peut créer ou rejoindre une salle. Le client peut envoyer les messages suivants pour gérer les salles :
  - `GG|CREATE_ROOM|nom_salle|max_joueurs|max_tentatives`
  - `GG|LIST_ROOMS`
  - `GG|JOIN_ROOM|nom_salle`
  - `GG|LEAVE_ROOM|nom_salle`
- Lorsqu'un message `GG|JOINED_ROOM` est reçu, le client établit une connexion directe avec les autres joueurs de la salle. Les communications pendant la partie se font alors en mode pair-à-pair. Lorsqu'un message `GG|GAME_STARTED` est reçu, les clients commencent la partie.
- Un joueur est désigné pour choisir la combinaison secrète. Le joueur qui choisit la combinaison informe les autres clients avec : `GG|SECRET_SET|nom_joueur` . Les autres joueurs peuvent alors envoyer des propositions sous la forme :
  - `GG|GUESS|couleur1|couleur2|couleur3|couleur4` Par exemple : `GG|GUESS|RED|BLUE|GREEN|YELLOW`

    - Le joueur détenteur de la combinaison secrète analyse la proposition et répond : `GG|FEEDBACK|couleurs_correctes|positions_correctes` Par exemple : `GG|FEEDBACK|3|1`
    - Si une proposition correspond exactement à la combinaison secrète, le joueur annonce la victoire avec : `GG|WINNER|nom_joueur`
- Une nouvelle partie peut être lancée dans la salle avec : `GG|NEW_GAME` .Les clients réinitialisent alors l'état du jeu et une nouvelle combinaison secrète est choisie.
- Si un client reçoit le message `GG|PLAYER_KICKED`, il doit quitter la salle et revenir à l'interface principale.

1- Travail demandé

Chaque équipe doit implémenter les parties client et serveur du protocole GG décrit dans les sections 2 et 3. La description fournie dans ce document couvre uniquement l'interaction entre le client et le serveur ; certaines décisions doivent être prises par les équipes. En ce qui suit, une liste non exhaustive d'exemples de telles décisions:

- Les descriptions de certaines interactions entre le client et le serveur ne couvrent pas tous les scénarios possibles. Les équipes sont invitées à travailler sous des hypothèses de leurs propre choix tant que ces hypothèses sont documentées dans les commentaires du code.
- Les interactions entre les clients durant une partie.
- La manière de fournir les configurations au client et au serveur : numéro de port, IP, etc.
- Etc.

À remettre :

- Un code qui implémente correctement les comportements du client et du serveur. L'exécution du code doit permettre à l'utilisateur de bien suivre l'exécution du programme. Autrement dit, vous êtes invités à afficher ou stocker les traces d'exécution du programme. Durant la démonstration, l'évaluation se fera à base des messages échangés entre le client et le serveur et les événements qui en découlent. Ces informations doivent être accessibles durant la démonstration pour permettre l'évaluation de l'implémentation. Des directives générales à propos des informations à afficher (chaque information dans une ligne différente):
    - Le type du message reçu.
    - Les différents champs du message et leurs valeurs.
    - Si la valeur d'un champ est une liste, affichez chaque élément de la liste sur une ligne distincte.
    - **Le client doit permettre l'envoi de n'importe quel type de message à n'importe quel moment.**
- Le code doit être écrit en **JAVA.**
- Un rapport **individuel** de trois pages qui contient les sections suivantes :
    - Section 1 : description du travail à faire (0.5 pages, 1.5%)
    - Section 2 : contribution individuelle à la réalisation du projet (0.5 pages, 1.5%)
    - Section 3 : Description détaillée d'une contribution à la réalisation du travail. Analyze, hypothèses, conception, validation. (1 page) (5%)
    - Section 4 : ce que vous avez appris de ce projet (0.5 pages) (3.5%)
    - Section 5 : ce que vous aurez pu faire autrement (conception ou implémentation) dans ce projet. (0.5 pages) (3.5%)
- Une démonstration de chaque équipe.

L’évaluation se fera à base de :

- Fonctionnement du code : en utilisant des cas de tests. (40%)
- Organisation du code : un code mal organisé sera pénalisé. (25%)
    - Le client et le serveur sont deux différents projets sur votre IDE. Deux différents jars doivent être exportés, un pour le client et un pour le serveur.
    - L’utilisation du multi-threading en conformité avec les bonnes pratiques vues en cours (exemple l’utilisation de join).
    - Organisation du code, chaque classe dans un fichier, répartition du code sur des méthodes. Par exemple un code du serveur qui contient seulement la classe main et méthode main sera fortement pénalisé dans cette partie de l’évaluation.
- La qualité de la présentation durant la démonstration. (10%)
- La qualité du rapport. (15%)
- Contribution individuelle. (10%)
- Des points bonus seront attribués pour : la qualité des tests utilisés par l’équipe pour tester son code, l’implémentation de la sécurité de l’échange entre le client et le serveur, l’implémentation de la tolérance aux pannes.